Ders 12 Genişletilmiş Notu: Cache Bellek: Bellek Hiyerarşisi, Yerellik İlkesi, Direct Mapped Cache, Ortalama Erişim Süresi ve Yazma Politikaları

Ders 12 Genişletilmiş Notu: Cache Bellek: Bellek Hiyerarşisi, Yerellik İlkesi, Direct Mapped Cache, Ortalama Erişim Süresi ve Yazma Politikaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Konulara Dair Sorular
- 2. Dersin Çerçevesi: Hit Oranı Neden Bu Kadar Yüksek?
- 3. Yerellik İlkesi
- 4. Bellek Hiyerarşisi
- 5. İşlemci ile Bellek Arasındaki Hız Farkı
- 6. Direct Mapped Cache: Basit Örnek
- 7. Adresin Alanlara Bölünmesi ve Blok Boyutu
- 8. 16 KB Direct Mapped Cache: Alan Hesabı
- 9. Cache Yükleme: Valid Bit, Hit, Miss ve Replacement
- 10. Direct Mapped Cache: Adım Adım Okuma Örneği
- 11. Ortalama Erişim Süresi
- 12. İki Seviyeli Cache
- 13. Yerine Koyma Algoritmaları
- 14. Yazma Politikaları: Write-Through ve Write-Back
- 15. Kapsamlı Örnek: Üç Eşleme Tekniğinde Adres Alanları

1. Önceki Konulara Dair Sorular

Yeni konuya geçmeden önce öğrencilerden önceki konularla ilgili iki soru geldi. İkisi de bir hatanın düzeltilmesiyle sonuçlandı.

1.1 Interface sorusu: adres uçlarının bağlanması

Soru: Interface problemlerindeki üçüncü soruda A7 ve A6 register select için ayrılıyor, kalan uçlar chip select'lere bağlanıyor. Ama metinde "address line 4", altıncıya bağlı deniyor. Sırayla değil de neden böyle bağlanıyor, yoksa biz mi yanlış anladık?

Cevap: Burada metinde bir hata vardır, oranın 5 olması gerekir. Hoca soruyu kitaptan aldığını, kitapta da aynı şekilde yazdığını belirtti ve "kitap da hata yapabilir" dedi.

Problemin kurulumu şöyledir. Derste görülen, içinde 4 tane internal register bulunan bir interface unit kullanılır. Adres çözümleme devresinde kapı konfigürasyonu kullanılmaz, adres uçları doğrudan bağlanır. Toplam 6 interface vardır ve adres yolu 8 bittir (A_0–A_7).

- **Register select uçları:** Her interface'in CPU tarafında bir chip select ve iki register select ucu (RS0, RS1) vardır. En yüksek anlamlı iki bit bütün interface'lere ortak gider: A_6 RS0'a, A_7 RS1'e.

- **Chip select uçları:** Kalan 6 uç (A_0–A_5) 6 interface'in chip select uçlarına tek tek bağlanır. A_0 birinci interface'e gider, sırayla devam eder ve A_5 altıncı interface'e gider.

Soruda bu konfigürasyon için bütün interface'lerin I/O alanındaki adreslerinin yazılması istenir. Burada dikkat edilecek nokta şudur: Bir interface'in I/O alanında tek adresi yoktur. İçinde 4 internal register olduğu için her interface'in 4 adresi vardır.

> **Ders sonrası not:** Hoca bağlantıyı anlatırken "A1 ucu birinci interface'in chip select ucuna bağlanacak" dedi. Doğrusu A_0'dır. Hocanın kendisinin de az önce söylediği gibi "address line 0" birinci interface'e, A_1 ikinci interface'e bağlanır.

1.2 Asenkron seri iletişim çerçevesi

Soru: Önceki derste bir çerçeve çizilirken başlangıçta da "stop" denmişti, sonunda da iki stop vardı. Çerçeve stop ile mi başlıyor?

Cevap: Hayır, çerçeve her zaman start biti ile başlar. Hoca bunun dalgınlıkla olduğunu söyledi ve hatayı düzeltti.

- **Start biti:** Her zaman standart tek bir bittir. Seviyesi lojik 0'dır.
- **Stop bitleri:** Seviyesi her zaman lojik 1'dir. Sayısı değişebilir: protokolü başta konuşurken 1 bit, 1.5 bit veya 2 bit tanımlanabilir. Bu ayrıntı Mikroişlemciler dersinde görülecek.

2. Dersin Çerçevesi: Hit Oranı Neden Bu Kadar Yüksek?

Bu derste geçen hafta başlanan cache memory konusu bütün ayrıntılarıyla bitiriliyor. Hoca önce cache'in bellek hiyerarşisindeki yerini, neden önemli olduğunu ve ne fayda sağladığını konuşacağını söyledi. Ardından üç eşleme tekniği (fully associative, direct mapped, set associative) yine ele alınacak, yazma teknikleri (write-through, write-back) ve yerine koyma politikası (cache replacement policy) anlatılacak.

Hocanın başta sorduğu soru şuydu: Cache'in hızlı bir bellek olduğunu biliyoruz, ama hit oranı nasıl bu kadar yüksek olabiliyor? Bir sistem aradığı veriyi %90 ya da %95 oranında cache'te bulabiliyor. Ana belleğin hangi adreslerini cache'e çekeceğimize nasıl karar veriyoruz da bu kadar başarılı oluyoruz? Bu sorunun altında yatan teori bir sonraki bölümün konusudur.

> **Hocanın vurgusu:** Bilgisayar Donanımı dersinin son dört haftası en önemli konuları anlatıyor. Giriş-çıkış ve iletişim, cache memory ve gelecek hafta anlatılacak virtual memory gerçekten önemlidir. Bu konular iyi öğrenilirse Mikroişlemciler dersinde de çok rahat edilir.
>
> **Hocanın vurgusu:** Cache memory konusunda hiçbir şeyi ezbere bilmek gerekmez. Konu dikkatli dinlenir ve matematiği takip edilirse cache memory ile ilgili her soru rahatça çözülebilir.

3. Yerellik İlkesi

Cache'in hit oranını bu kadar yüksek yapan şey yerellik ilkesidir (principle of locality). Cache memory teknolojisi bu kavrama dayanarak geliştirilmiştir.

Mantığı şöyledir. Yazdığınız programın arka arkaya gelen komutları, birbirini takip eden bellek adreslerine yerleştirilir. Kodun içinde döngüler (for loop) ve belli sıklıkla gidip çalıştırılması gereken ortak alt programlar (subroutine) bulunabilir. Bu durumda program çalışırken ana belleğin belli bir bölgesine odaklanmış olursunuz, yani belli bir adres alanının içinde gezersiniz.

Benzer durum veri için de geçerlidir. Bellekte bir look-up table, büyük bir matris, bir dizi veya bir tablo olabilir. Bu yapıların bulunduğu bellek alanına belki yüz, iki yüz defa okuma yapmak gerekebilir. Bu okumalar ya aynı adreslerden ya da birbirine yakın adreslerden yapılır. Sonuçta yerellik, bölgesellik gösteren bir adres bölgesinin içinde sürekli gezilir.

Bu ilke iki şekilde tanımlanır:

- **Zamansal yerellik (temporal locality):** *Same location will be referenced again soon.* Aynı adrese yakın zamanda tekrar erişilebilir. Örneğin bir adresi okudunuz, üç adım sonra

aynı adresi tekrar okuyabilirsiniz. Bir döngünün içinde x'i bir artırmak gibi bir işlem buna örnektir. Bu, zamana bağlı bir yerelliktir.

- **Uzamsal yerellik (spatial locality):** *Nearby locations will be referenced soon.* Bir bellek adresine erişildiyse, bunun bir üstündeki veya altındaki adreslere de yakın zamanda erişilebilir. Bu, bulunduğu yere bağlı bir yerelliktir.

Cache tasarlanırken uzamsal yerellik şöyle desteklenir: Veri tek bir adres olarak değil, bir blok olarak taşınır. Böylece erişilen adresin üstündeki ve altındaki adres gözleri de cache'e gelmiş olur.

4. Bellek Hiyerarşisi

Hoca cache'in bütün bellek hiyerarşisi içindeki yerini bir slaytla gösterdi. CPU yongasının içinde internal register'lar ve birinci seviye cache (L1) vardır. Yonganın dışında, anakart üzerine yerleştirilmiş ikinci seviye cache (L2) bulunur. Ardından DRAM entegrelerinden oluşan ana bellek ve disk gelir. Her birim için kapasite, hız, maliyet (megabyte başına kaç dolar) ve blok boyutu incelenir.

Slaytın değerleri eski bir mimari yapıya aittir. Hocaya göre önemli olan değerlerin kendisi değil, mertebelerdir, yani birimlerin birbirinin kaç katı hızda olduğudur.

Seviye	Derste söylenen değer
Register	Erişim 1.4 ns. Veriye en hızlı erişilen yer.
L1 cache	Erişim 4.2 ns, kapasite 128 KB.
L2 cache	L1'den 4 kat yavaş, kapasitesi daha büyük. Maliyet megabyte başına 90 dolar.
Ana bellek	L2'nin neredeyse 5 katı yavaş. Hoca özellikle hızına dikkat çekti.
Disk	En büyük kapasiteli, en yavaş ve en ucuz seviye.

> **Ders sonrası not:** Hoca L1 süresi için "bunun 3 katı, yaklaşık 4 katı" dedi. $4.2/1.4 = 3$ olduğu için L1, register'dan 3 kat yavaştır.
>
> **Ek bilgi (derste anlatılmadı):** Slaytta L2'nin anakart üzerinde gösterilmesi eski mimariye özgüdür. Günümüz işlemcilerinde L2, çoğu zaman L3 de, büyük olasılıkla işlemci yongasının içindedir.

Slaytın ana fikri şudur: İşlemciden uzaklaşıp sağa doğru gidildikçe kullanılan bellek teknolojilerinin kapasitesi büyür, hızı yavaşlar ve maliyeti ucuzlar. Örneğin cache, DRAM ve diskle karşılaştırıldığında daha pahalıdır.

Hoca hiyerarşiyi bir piramit olarak da gösterdi. İşlemci piramidin tepesindedir. Ona yakın olan birimlere level 1, uzaklaştıkça level 2, level 3 denir. Slaytta *increasing distance from processor, decreasing cost* yazılıdır, yani uzaklaştıkça maliyet düşer. Yukarısı *higher levels*, aşağısı *lower levels* olarak adlandırılır. Piramidin tabanı ne kadar genişse o seviyenin kapasitesi de o kadar büyüktür.

5. İşlemci ile Bellek Arasındaki Hız Farkı

Hoca burada Moore kanunundan söz etti. Anlatımına göre firmanın ürettiği teknolojileri inceleyen bu kişi şu tespitte bulunuyor: Üretilen mikroişlemcilerin hızı her yıl %60 artıyor, ama ana bellekte kullanılan DRAM entegrelerinin erişim hızı her yıl yalnızca %9 iyileşiyor.

Oysa mikroişlemci ile ana bellek birlikte düşünülmelidir. Sistem ancak ikisi birlikte hızlı çalışırsa hızlanabilir. Her geçen yıl ikisi arasındaki açıklık giderek artıyor, bu yüzden bellek teknolojilerinde acil bir çözüm gerekiyor.

> **Ders sonrası not:** Hoca bu kişiyi "Intel'de çalışan bir iktisatçı, işletmeci" diye anlattı. Gordon Moore büyük olasılıkla Intel'in kurucularından bir kimyagerdir. Moore Yasası da büyük olasılıkla bir yongadaki transistör sayısının yaklaşık iki yılda bir ikiye

katlanmasını anlatır. İşlemci ile DRAM hızlarının karşılaştırması büyük olasılıkla bu yasanın değil, işlemci-bellek performans açığı grafiğinin konusudur.

Bu açığı kapatmak için iki yola başvuruldu:

- **Veri yolunu genişletmek:** Veri yolu 16 bitten 32 bite, 32 bitten 64 bite çıkarıldı. Böylece ana bellekten tek erişimde 64 bit okumak mümkün hale geldi, aynı veri için iki kez okumak gerekmedi.
- **Cache memory:** Bunun yanına ilave olarak cache teknolojisi girdi.

  Ders sonrası not: Hoca 64 bit için "4 byte'ı bir seferde okuyorsunuz" dedi. $64/8 = 8$ olduğu için tek erişimde okunan miktar 8 byte'tır. 4 byte, 32 bite karşılık gelir.

Soru: Cache'ler adresi nasıl tutuyor? Pointer gibi adresi mi gösteriyorlar, yoksa adresi kopyalıyorlar mı?

Cevap: Adresin tamamı değil, belli bir bölümü kopyalanır. Fully associative yapıda neredeyse adresin tamamı kopyalanır diyebiliriz. Veri kaydedilirken adres bilgisi de veriyle birlikte oraya konur, yani cache üzerinde bu adres bilgisini tutmak için fiziksel bir alan ayrılır.

6. Direct Mapped Cache: Basit Örnek

Hoca üç eşleme tekniğini hatırlattı ve önce direct mapped cache'i anlatan, kendi deyişiyle "çocuk örneği gibi" basitleştirilmiş bir şekil gösterdi. Ana bellek 16 satırdan (16 byte), cache ise 4 satırdan (4 byte) oluşuyor. Cache ana bellekten çok daha küçük olduğu için ana belleğin yalnızca belli bölgeleri buraya taşınabilir.

Adresler 0000 ile 1111 arasındadır. Direct mapped yapıda cache'in her satırı, ana bellekte yalnızca adresinin en düşük anlamlı iki biti kendi index'ine eşit olan satırları tutabilir. Bu yüzden her index için ana bellekte 4 adres vardır.

Cache index'i	Bu satırı kullanabilen ana bellek adresleri
00	0000, 0100, 1000, 1100
01	0001, 0101, 1001, 1101
10	0010, 0110, 1010, 1110
11	0011, 0111, 1011, 1111

Derste slaytta 00 index'li satır kırmızıyla, 11 index'li satır kahverengiyle gösterilerek bu iki satır anlatıldı. Tabloda 01 ve 10 satırları aynı kuraldan hesaplanarak eklendi (ara adım eklendi).

Bu örnekte veri tek byte olarak taşınıyor. Tek byte taşımak yalnızca zamansal yerelliği destekler. Gerçekte veri blok olarak taşınacaktır: Referans verilen adresin üstünde ve altında kalan adresler de bloğun içinde gelir. Böylece uzamsal yerellik de desteklenir ve hit oranı artar.

7. Adresin Alanlara Bölünmesi ve Blok Boyutu

Direct mapped cache'te adres yolundan gelen bilgi alanlara bölünür. Slaytta yalnızca *byte offset* yazıyordu, ama derste bu alan ikiye ayrılıyor. Yüksek anlamlı bitten düşük anlamlı bite doğru alanlar şunlardır:

Tag	Index	Word offset	Byte offset

- **Index:** Cache'teki satır numarasını gösterir. Kaç bit olacağına cache'in satır sayısına göre karar verilir.
- **Offset:** Erişilmek istenen adresi bloğun içinde word bazında, word'ün içinde byte bazında gösterir.
- **Tag:** *Remaining bits after offset and index are determined.* Index ve offset belirlendikten sonra geriye kalan bitlerdir.

Tag alanının cache'e mutlaka yazılması gerekir. Az önceki sorunun cevabı da budur: Adres bilgisinin cache'te bulunması gerekir, ama adresin yalnızca tag alanı tutulur. Çünkü diğer alanlar index ve offset için zaten ayrılmıştır.

Uzamsal yerelliği desteklemek için veri bir word olarak değil, örneğin 4 veya 8 word'lük bloklar hâlinde taşınır. Slayttaki grafikte blok boyutunu artırmanın miss oranını düşürdüğü görülür. Ancak bu düşüş belli bir noktadan sonra durağanlaşır, sonra da eğri yukarı kıvrılmaya başlar. Hoca bunun sebebini, cache kapasitesinin sınırlarının zorlandığı bir noktaya gelinmesi olarak açıkladı.

> **Ek bilgi (derste anlatılmadı):** Kapasite sabitken blok büyüdükçe satır sayısı azalır, çünkü satır sayısı kapasitenin blok boyutuna bölümüdür. Daha az satır, farklı adres bölgelerinin aynı satırlara daha sık çakışması demektir. Miss oranının yeniden artması büyük olasılıkla bundan kaynaklanır.

Soru: Tag'i de cache'te tutuyoruz dediniz. Bir örnekte cache alanı 2 MB olarak verilmişti. Bu alan yalnızca içindeki veri için mi?

Cevap: Evet, yalnızca veri için. Tag için ayrılmış alanlar hesaba katılmaz. Onlar teknoloji üretilirken ayrıca dahil edilir, ama hesaplarda cache kapasitesi dediğimizde her zaman veri için kullanılan kapasite anlaşılır.

8. 16 KB Direct Mapped Cache: Alan Hesabı

Örnek şöyledir: *Suppose we have 16 KB of data in a direct mapped cache with 4-word blocks. Determine the size of tag, index and offsets if we are using 32-bit architecture.*

"32 bit mimari" denmesi, adres yolunun da veri yolunun da 32 bit olduğu anlamına gelir. Hoca bunların genelde birbirinden farklı olabileceğini ve ayrı ayrı verildiğini, burada tek ifadeyle verildiğini ekledi.

> **Hocanın vurgusu:** Bu önemli bir noktadır. Bir word, mimarinin veri yolu genişliği (data bus width) kadardır. Burada veri yolu 32 bit olduğu için 1 word = 32 bit = 4 byte'tır.

Verilen: Adres 32 bit, veri yolu 32 bit, cache kapasitesi 16 KB, blok 4 word.

Blok boyutu:

$$1 \text{ blok} = 4 \text{ word} \times 4 \text{ byte} = 16 \text{ byte} = 2^4 \text{ byte}$$

Satır sayısı: Cache'in her satırına bir blok yerleşir, yani bir satır 16 byte'tır. Toplam kapasiteyi bir satırın kapasitesine bölünce satır sayısı (*number of rows per cache*) bulunur:

$$\frac{16 \text{ KB}}{16 \text{ byte}} = \frac{2^{14} \text{ byte}}{2^4 \text{ byte}} = 2^{10} \text{ satır}$$

Bu satırları adreslemek için gereken index biti 10'dur.

Byte offset: 1 word 4 byte olduğu için bunu adreslemek için 2 bit gerekir. Bir word'ün içinde byte byte okuma yapılmak istenebilir: en düşük anlamlı byte okunabilir ya da arka arkaya gelen 2 byte (16 bit) okunabilir. Bu bit alanı bunun içindir.

Word offset: Blok boyutuyla ilişkilidir. Bir blok 4 word'den oluştuğu için bloğun içindeki her word'e erişmek için 2 bit gerekir.

Tag: Offset toplam 4 bit, index 10 bit, ikisi birlikte 14 bit eder. Geriye kalan bitler tag'dir:

$$\text{tag} = 32 - 10 - 2 - 2 = 18 \text{ bit}$$

Tag	Index	Word offset	Byte offset
18 bit	10 bit	2 bit	2 bit

9. Cache Yükleme: Valid Bit, Hit, Miss ve Replacement

Başlangıçta cache'in bütün satırları boştur. Bir satırda geçerli veri olup olmadığını gösteren valid bit vardır.

- **Valid bit:** 1 ise cache'in o gözü doludur, geçerli veri vardır. 0 ise göz boştur.
- **Cache hit:** Referans verilen adres cache'te vardır, erişim isabetlidir. Slayttaki tanımı *cache block is valid and contains proper address* şeklindedir. Yani valid biti 1 olacak ve o satırda tutulan tag ile erişilmek istenen adresin tag'i birbirini tutacaktır.
- **Cache miss:** Bu koşullar sağlanmıyorsa aranan veri cache'te yoktur.
- **Replacement:** Miss olduğunda ana bellekten okunan veri aynı zamanda cache'e kopyalanır. Yani blok alınıp cache'e yerleştirilir. Hoca replacement'ı, daha önce taşınmış bir verinin üzerine yazmak (override) olarak tanımladı.

Soru: Replacement sırasında ekstra bir işlem yapıyor muyuz, bunun bir maliyeti var mı? Ana bellekten veri getirirken yol üstünde cache'e kaydedip oradan akümülatöre geçilebilir gibi görünüyor. Bu kopyalama hız açısından ek maliyet doğuruyor mu?

Cevap: Hayır. Maliyet, aranan veriyi cache'te bulamayıp ana bellekten getirmenin maliyetidir. Cache'e kaydetme süresiyle ilgili bir zaman kaybı hesaba katılmaz. Karar verilmesi gereken konu ise başkadır: Örneğin set associative bir yapıda hangi sete kopyalamanın daha iyi olacağına, hangisinin daha az kullanıldığına karar vermek gerekir. Replacement policy burada devreye girer.

> **Ek bilgi (derste anlatılmadı):** "Kayıt süresi hesaba katılmaz" ifadesi dersteki basit hesap modelinin varsayımıdır. Gerçek sistemlerde blok yükleme, write-back kullanılıyorsa dirty bloğun geri yazılması da, büyük olasılıkla miss cezasının bir parçası sayılır.

10. Direct Mapped Cache: Adım Adım Okuma Örneği

Hoca bu örneği konuyu çok net anlatan güzel bir örnek olarak tanıttı ve dikkatle dinlenmesini istedi. Yapı önceki bölümdeki 16 KB'lık cache'tir: tag 18 bit, index 10 bit, word offset 2 bit, byte offset 2 bit.

Ana bellekte her word 4 byte'lık bir değişken tutar, çünkü veri yolu 32 bittir. Adresler hexadecimal verilmiştir. Bir kısmı şöyledir:

Adres	0x10	0x14	0x18	0x1C	0x30	0x34	0x38	0x3C
Değişken	a	b	c	d	e	f	g	h

İşlemci sırayla dört adresten okuma yapar: 0x14, 0x1C, 0x34 ve 0x8014. Adresler 8 hex basamak, yani $8 \times 4 = 32$ bit olarak verilir. Binary'ye açılıp alanlara bölündüğünde şu tablo çıkar:

Adres	Tag (18 bit)	Index (10 bit)	Word off.	Byte off.
0x00000014	000000000000000000	0000000001	01	00
0x0000001C	000000000000000000	0000000001	11	00
0x00000034	000000000000000000	0000000011	01	00
0x00008014	000000000000000010	0000000001	01	00

Cache'in index'i 0'dan 1023'e kadar değer alır, çünkü 10 bitte ifade edilebilen aralık budur. Her satırda 1 bitlik valid gözü, 18 bitlik tag gözü ve 4 word'lük veri alanı vardır. Veri alanındaki word'lerin blok içindeki adresleri şöyledir:

- Word 0: 0–3
- Word 1: 4–7
- Word 2: 8–B
- Word 3: C–F

Başlangıçta bütün valid bitleri 0'dır.

10.1 Birinci okuma: 0x14

Index 1 olduğu için cache'in 1 numaralı satırına gidilir. Valid biti 0'dır, yani bu satırda geçerli veri yoktur (*no valid data*). Sonuç **cache miss**'tir.

İşlemci ana bellekten okur. Bu daha maliyetlidir, daha fazla zaman gerektirir. Okuduğu veriyi CPU içindeki internal register'lara taşırken aynı zamanda cache'e de kaydeder. Valid biti 1 yapılır, tag 0 yazılır. Word offset 1 olduğu için aranan b değişkeni Word 1'e gelir.

İşlemcinin okumak istediği yalnızca b'dir. Ama veri blok olarak taşındığı için komşuları a, c ve d de cache'e gelir. Uzamsal yerellik der ki: Word 1'den okuyorsanız, yakın zamanda komşu adreslerden de okuma ihtiyacınız olabilir. Bu bir hazırlıktır.

Soru: Slaytta offset neden 4 bit görünüyor?

Cevap: Offset ikiye ayrılır: byte offset ve word offset. Bu kaynak ikisini birlikte kullanıyor, ayrı göstermiyor. Biz ayrı ayrı gösteriyoruz.

10.2 İkinci okuma: 0x1C

Index yine 1'dir. 1 numaralı satıra gidilir. Valid biti 1'dir, yani geçerli veri vardır. Sonra erişilen adresin tag'i ile satırdaki tag karşılaştırılır. İkisi de 0'dır, eşleşirler. Sonuç **cache hit**'tir.

Word offset 3 olduğu için aranan veri Word 3'tedir. d değişkeni cache'ten okunur. Bu hit, 0x14 okunurken bütün bloğun gelmiş olmasının sonucudur.

10.3 Üçüncü okuma: 0x34

Bu adımı hoca öğrencilere sordu.

Soru: 0x34 için sistem nasıl çalışır?

Cevap: Index 3 olduğu için 3 numaralı satıra gidilir. Valid biti 0'dır. Sonuç **cache miss**'tir, çünkü geçerli veri yoktur. Veri ana bellekten cache'e taşınır. Okunmak istenen Word 1'deki f değişkenidir, ama blok olarak komşuları e, g ve h de gelir.

10.4 Dördüncü okuma: 0x8014

Bu adımı da hoca öğrencilere sordu.

Soru: 0x8014 için sonuç nedir?

Cevap: Bir öğrenci, index 1 ve valid biti 1 olduğu için hit olduğunu söyledi. Hoca "daha hit olmadı, bir şeye daha bakman lazım" diyerek düzeltti. Geçerli veri olması yetmez, tag'ler de karşılaştırılmalıdır. Erişilmek istenen adresin tag'i 2, cache'te tutulan verinin tag'i 0'dır. *Tag does not match*. Sonuç **cache miss**'tir.

Veri yine ana bellekten cache'e taşınır. Yeni adresin tag'i 2 satıra kaydedilir. Yani replacement yapılmıştır: Önceden getirilen a, b, c, d bloğunun üzerine, yeni adres bölgesinin bloğu yazılmıştır.

Dört okumanın özeti:

Adres	Tag	Index	Word offset	Valid (önce)	Sonuç	Okunan
0x14	0	1	1	0	Miss, a b c d bloğu 1. satıra	b
0x1C	0	1	3	1	Hit	d
0x34	0	3	1	0	Miss, e f g h bloğu 3. satıra	f
0x8014	2	1	1	1	Miss (tag $0 \neq 2$), 1. satır değişir	0x8014'teki word

Bu dizide 4 erişimden 1'i hit olmuştur, isabet oranı $1/4 = 0.25$'tir (ara adım eklendi).

10.5 Direct mapped'de seçim yoktur

Direct mapped cache'te başka alternatif yoktur. İndeksi 1 olan tek bir satır bulunduğu için veri oraya yazılmak zorundadır.

Soru: En az kullanılan gözü silip üzerine yazan bir yöntemden bahsetmiştiniz. Buna nasıl karar veriliyor?

Cevap: Bu yöntem set associative yapıda kullanılır. İki ya da dört set olsaydı kullanılırdı. Direct mapped cache'te alternatif bir göz yoktur: Index neyi gösteriyorsa o satıra yazmak zorundasınız. Four-way set associative bir cache düşünelim. Aynı index'e sahip dört göz vardır ve hangisine yazılacağına karar vermek gerekir. Bunun için LRU (least recently used) biti ya da clock denilen bir gösterge kullanılır. Bu gösterge yakın zamanda kullanımı günceller, böylece yakın zamanda kullanılmayan set bulunup değiştirilir.

> **Ders sonrası not:** Hoca bu cevabın ortasında "direct map cache'te ne yapıyorduk? Aynı indekse sahip dört set var" dedi. Burada kastedilen set associative cache'tir. Hemen ardından kendisi de four-way set associative yapıyı örnek verdi ve direct mapped'de alternatif göz olmadığını yineledi.

Hoca bu noktada bir döngü içinde iki dizi üzerinde işlem yapan *vector product* örneğini geçti. Bu örnek cache'in sağladığı avantajları gösteriyor ve incelemesi öğrencilere bırakıldı.

11. Ortalama Erişim Süresi

Cache memory'de ortalama erişim süresi (average access time) hesaplanabilir. Semboller şunlardır:

- T_a: Ortalama erişim süresi (*average access time*)
- T_c: Cache erişim süresi (*cache access time*)
- T_m: Ana bellek erişim süresi (*main memory access time*)
- H: Hit oranı (*hit ratio*), $1 - H$: miss oranı

Formül şöyle yazılır:

$$T_a = H \cdot T_c + (1 - H)(T_c + T_m)$$

Miss durumunda $T_c + T_m$ alınmasının sebebi şudur: Miss olduğunu anlamak için önce cache'e bakılır, sonra ana belleğe gidilir. Cache'te yalnızca valid bit ve tag karşılaştırması yapılır, ama bunun maliyeti eklenir. Üstüne ana bellekten getirme süresi konur.

Parantez açılıp sadeleştirilince:

$$T_a = HT_c + T_c + T_m - HT_c - HT_m = T_c + (1 - H)T_m$$

Örnek. Verilen: $H = 0.85$, $T_m = 50$ ns, $T_c = 5$ ns. Hoca bu mertebelere dikkat çekti: Cache erişimi gerçekten 5 ns civarındadır, ana bellek genelde bunun 10 katı kadar sürer.

$$T_a = T_c + (1 - H)T_m = 5 + 0.15 \times 50 = 5 + 7.5 = 12.5 \text{ ns}$$

Ana bellek erişim süresi olan 50 ns ile karşılaştırıldığında çok iyi bir kazanç vardır. Ortalama süre 4 kat kısalmıştır, çünkü $50/12.5 = 4$ (ara adım eklendi).

12. İki Seviyeli Cache

Hoca iki seviyeli cache için bir örnek daha çözdü.

Verilen:

Seviye	Hit süresi	Miss oranı
L1	1 cycle	%5
L2	5 cycle	%15
Ana bellek	100 cycle	—

L1'in miss oranı %5 demek, verinin %95'i L1'de bulunuyor demektir. L2'de bulunamazsa ana belleğe gidilir, yani L2'nin miss cezası (miss penalty) DRAM'den okumanın maliyeti olan 100 cycle'dır.

Birinci adım, L1 miss cezası: L2'den getirmenin maliyetidir. L2'nin erişim süresine, L2'de bulamama ihtimali çarpı ana bellekten getirme maliyeti eklenir.

$$\text{L1 miss cezası} = 5 + 0.15 \times 100 = 5 + 15 = 20 \text{ cycle}$$

İkinci adım, ortalama erişim süresi: L1'in erişim süresine, L1 miss oranı çarpı L1 miss cezası eklenir.

$$\text{Ortalama erişim süresi} = 1 + 0.05 \times 20 = 1 + 1 = 2 \text{ cycle}$$

Hocaya göre iki seviyeli hesap biraz karışık görünse de çok kompleks değildir.

13. Yerine Koyma Algoritmaları

Hoca yerine koyma algoritmalarını kısaca geçti.

- **LRU (least recently used):** Daha önce anlatıldığı gibi kullanım bilgisine dayanır.
- **FIFO (first in, first out):** Kullanım sıklığı gibi bir bilgi üzerinden gitmeyen (*non-usage based*) bir çözümdür.
- **Random:** Rastgele seçim yapılır. Bu da kullanım bilgisine dayanmaz.
- **Optimal algoritma:** Slaytta yukarıda yer alır. Hoca bunun da kullanıma dayalı (*usage based*) olduğunu hatırladığını söyledi.

  Ek bilgi (derste anlatılmadı): Optimal algoritma büyük olasılıkla, gelecekte en geç kullanılacak bloğu çıkaran algoritmadır. Gelecekteki erişimleri bilmeyi gerektirdiği için büyük olasılıkla pratikte değil, diğer algoritmalarla karşılaştırma için kullanılır.

14. Yazma Politikaları: Write-Through ve Write-Back

Şimdiye kadar hep okumadan söz edildi: İşlemci önce cache'e bakar, bulamazsa ana bellekten okur. Yazma işleminde işler farklıdır.

Bir incelemeye göre, bütün bilgisayar operasyonlarında işlemci ortalama %75 okuma, yalnızca %25 yazma yapar. Okuma işleminde cache kullanarak çok büyük verim elde edilir. Yazma işleminde bu verim biraz düşer. Yazma için iki teknik kullanılır.

14.1 Write-Through

On cache write, always update main memory as well. Yazılacak adres cache'te olsun ya da olmasın, cache'e yazma yapıldığı anda aynı veri ana belleğe de yazılır. Sebebi şudur: Cache, ana belleğin küçük bir bölgesinin kopyasını tutar. Bu ikisinin tutarlı olması için write-through her iki alanı birden günceller.

Donanım yapısında işlemci, cache ve ana bellek vardır. Okumada yükleme (load) ana bellekten cache'e, cache'ten işlemciye doğrudur. Yazmada (store) ise her iki bloğa aynı anda store yapılır. Bu teknikte *memory always consistent with cache*: Ana bellek ile cache her zaman tutarlıdır, aynı adreste aynı veriyi gösterir.

14.2 Write-Back

On cache write, remember that block is modified. Cache'te hangi bloğun değiştiğini hatırlamak için bir **dirty biti** kullanılır. Bunun için her satırda fazladan bir bit ayrılır.

- **Dirty biti 0:** Cache'teki veri ile ana bellekteki veri aynıdır.
- **Dirty biti 1:** Cache'teki veri güncellenmiştir, ama ana bellekteki kopya güncel değildir.

Amaç, sık sık gidip ana belleğe yazma maliyetinden kurtulmaktır. Ana bellek yalnızca cache'in ilgili satırı değiştirileceği zaman güncellenir. O satıra başka bir veri kaydedileceği zaman önce dirty bitine bakılır. Dirty biti 1 ise satırdaki veri önce ana belleğe yazılır, sonra üzerine yazılır (*update main memory when dirty block is replaced*).

Donanım yapısında store önce cache'e yapılır. Ana belleğe ise cache üzerinden, write-back ile geçer.

Özellik	Write-through	Write-back
Yazma anında	Cache ve ana bellek birlikte	Yalnız cache
Ana belleğin güncellenmesi	Her yazmada	Dirty blok değiştirilirken
Ek bit	Dersteki anlatımda yok	Dirty biti
Tutarlılık	Her zaman tutarlı	Dirty blokta ana bellek eski kalır

14.3 Write-through örneği ve LRU'nun çalışması

Hoca, write-through'yu ve LRU bitinin güncellenmesini gösteren bir simülasyon slaytı açtı. Solda işlemcinin çalıştırdığı 5 komut ve R0–R3 internal register'ları vardır. Ortada iki satırlık cache vardır, her satırın bir tag'i vardır ve iki byte ya da iki word tutar. Sağda 0'dan 15'e kadar adreslerdeki verilerle ana bellek vardır. Slaytta yazmasa da hoca bu yapının fully associative olduğunu hatırladığını söyledi.

Fully associative yapıda yazma index'e bağlı olmadığı için veri iki satırdan herhangi birine gidebilir. Blok, boş olan göze ya da en az kullanılan satıra taşınır. LRU biti de bunu gösterir.

1. **Ana bellek 1 adresinden R1'e okuma:** Bu adreste 29 vardır. Cache başta boş olduğu için ilk miss oluşur. İşlemci tek bir değer istese de blok taşındığı için 78 ve 29 birlikte cache'e gelir. R1'e yalnızca 29 alınır. O satırın valid biti 1 yapılır. LRU, alttaki satırı en az kullanılan, üstteki satırı yakın zamanda kullanılan olarak gösterir.
2. **Ana bellek 7 adresinden okuma:** Adres 0111'dir. 162 ve 173 sayıları bir sonraki boş satıra yerleşir. Bu satırın da valid biti 1 olur, miss sayısı 2'ye çıkar. En son alt satıra yerleşme yapıldığı için artık LRU üstteki satırı gösterir.
3. **R2'nin ana bellek 0 adresine yazılması (store):** Hoca bu noktada, göstermek istediğinin LRU'nun sürekli yer değiştirmesi olduğunu söyleyip örneği burada bıraktı. Devamı öğrencilere bırakıldı.

   Ders sonrası not: Hoca ikinci adımda tag'in 0111 olduğunu, yani adresin tamamının tag olarak kullanıldığını söyledi. Bir satırda iki birim tutulduğu için adresin en düşük biti satır içindeki konumu seçer. 4 bitlik 0111 adresinde bu durumda tag 011, offset 1'dir ve blok 6 ile 7 adreslerini kapsar. 1 adresi için de tag 000'dır. Hocanın daha önce söylediği "neredeyse adresin tamamı" ifadesi doğrudur: Fully associative yapıda tag, adresten offset çıkarılarak bulunur.

Hoca LRU'nun yalnız fully associative yapıda olmadığını da ekledi: Birden fazla set varsa aynı mantık set associative yapıda da kullanılabilir. Aynı örneğin write-back sürümü de slaytlarda vardır, nasıl çalıştığı oradan incelenebilir.

15. Kapsamlı Örnek: Üç Eşleme Tekniğinde Adres Alanları

Hoca dersi kapsamlı bir örnekle bitirdi.

Suppose a microprocessor that physical addresses are 32 bit wide and data bus is 64 bit. Cache'in toplam kapasitesi 2 MB (not including tag bits), line size 32 byte.

Hoca, “not including tag bits” ifadesinin her seferinde yazılmak zorunda olmadığını belirtti. Cache kapasitesinden söz edilirken tag alanları zaten dahil edilmez. *Line size* demek *block size* demektir.

Ortak hesaplar:

- Adres yolu: 32 bit.
- 1 word = veri yolu genişliği = 64 bit = 8 byte.
- 1 line = 32 byte = $32/8 = 4$ word. Yani bir satırda Word 0, Word 1, Word 2, Word 3 vardır.
- Byte offset: $\log_2 8 = 3$ bit.
- Word offset: $\log_2 4 = 2$ bit.
- Toplam offset: 5 bit.

15.1 Fully associative

Index alanı kullanılmaz. Adres yalnız tag, word offset ve byte offset alanlarından oluşur.

$$\text{tag} = 32 - 5 = 27 \text{ bit}$$

15.2 Direct mapped

Byte offset ve word offset aynı kalır, ama index alanı gerekir. Index'in kaç bit olacağına cache'in satır sayısı bilinerek karar verilir (*number of rows in cache*). Bir satırda bir line kadar, yani 32 byte veri tutulur.

$$\frac{2 \text{ MB}}{32 \text{ byte}} = \frac{2^{21} \text{ byte}}{2^5 \text{ byte}} = 2^{16} \text{ satır}$$

Index 16 bittir. Tag ise kalan bitlerdir:

$$\text{tag} = 32 - (16 + 2 + 3) = 32 - 21 = 11 \text{ bit}$$

> **Ders sonrası not:** Hoca bu hesapta “32 MB, 2 üzeri 5 MB” dedi. Doğrusu 32 byte $= 2^5$ byte'tır. Kapasite de 2^{21} byte'tır. Hoca ardından “byte'lar kısaldı” diyerek hesabı byte üzerinden sürdürdü ve sonuç doğru çıktı.

Hoca bu bölümlemenin nerede işe yarayacağını da hatırlattı: Bu soruda yalnızca alanların kaç bit olduğu soruluyor. Ama arka arkaya gelen adresler hexadecimal verilip bu adreslerdeki değişkenler a, b, c, d diye verilseydi, 10. bölümdeki örnekte olduğu gibi cache'e nasıl yerleştirileceği de artık bu bölümlemeden bulunabilirdi.

15.3 Two-way set associative

Hoca two-way set associative yapıyı, cache'i iki sete bölmek olarak anlattı. Toplam kapasite yine 2 MB'tır, set 0 ve set 1'in her birinde 32 byte'lık bloklar yer alır. Blokların kendi tag alanları vardır. Şekilde tag alanı çizilmese de durum aynıdır, toplam kapasite dediğimiz veri için ayrılan kapasitedir.

Satır sayısı hesaplanırken toplam kapasite iki sete bölünür:

$$\frac{2^{21}/2 \text{ byte}}{2^5 \text{ byte}} = \frac{2^{20}}{2^5} = 2^{15} \text{ satır}$$

Index 15 bittir. Tag ise kalan bitlerdir:

$$\text{tag} = 32 - (15 + 2 + 3) = 12 \text{ bit}$$

> **Ders sonrası not:** Hoca two-way yapının şekli üzerinde “tag alanı bu cache için her bir satırda 11 bit” dedi. 11 bit direct mapped sonucudur. Two-way yapıda hesap 12 bit verir, hoca da hemen sonra 12 bit buldu.

> **Ders sonrası not:** Hoca “set 0, set 1” derken cache’in iki parçasını kastediyor. Bu kullanım derste böyledir. Yaygın terminolojide ise büyük olasılıkla bu parçalara *way*, aynı index’e sahip satırların oluşturduğu gruba *set* denir. Bu adlandırmayla 2^{15} sayısı set sayısı olur. Bit hesapları değişmez.

Hoca aynı yapının four-way set associative olarak da benzer şekilde tasarlanabileceğini söyledi.

Sonuçların özeti:

Teknik	Tag	Index	Word offset	Byte offset
Fully associative	27	—	2	3
Direct mapped	11	16	2	3
2-way set associative	12	15	2	3

15.4 Word boyutu sorusu

Soru: Sorunun başında word boyutunu nasıl bulduğumuzu anlamadım. Veri yolu genişliği doğrudan bir word mü oluyor?

Cevap: Evet. 8086 Assembly dersinde işlemcinin veri yolu 16 bit olduğu için bir word’ün hep 16 bit olduğunu öğrendiniz. Bu mimari yapıda word, işlemcinin veri yolu kadardır. 32 bitlik işlemcide bir word 32 bit, 64 bitlik veri yoluna sahip işlemcide 64 bittir.

> **Hocanın vurgusu:** Word konusunda her sene öğrencilerin kafası karışıyor. *Data bus width* ne kadarsa bir kelime o kadardır.

Soru: Ama byte’ın boyutu değişmiyor, bir byte 8 bit. Hesap oradan mı çıkıyor?

Cevap: Evet, ama byte offset alanını belirleyen şey veri yolu genişliğidir. Bu yapıda veri yolu 8 byte’tan oluşuyor. Bu 8 byte’ı byte byte adreslemek için 3 bit gerekir.

Dersin Toparlaması

Cache’in yüksek hit oranı yerellik ilkesinden gelir. Programlar ve veriler belli adres bölgelerinde yoğunlaşır: Aynı adrese yakın zamanda yine erişilir (zamansal yerellik), yakın adreslere de yakın zamanda erişilir (uzamsal yerellik). Veri blok olarak taşındığı için ikisi de desteklenir. Bellek hiyerarşisinde işlemciden uzaklaştıkça kapasite artar, hız ve maliyet düşer. İşlemci ile DRAM arasında büyüyen hız farkı cache’i gerekli kılar.

Adres tag, index, word offset ve byte offset alanlarına bölünür. Word, veri yolu genişliği kadardır. Satır sayısı kapasitenin blok boyutuna bölümüdür. Hit için valid bitin 1 olması ve tag’lerin eşleşmesi gerekir. Performans $T_a = T_c + (1 - H)T_m$ ile ölçülür, iki seviyede de aynı mantık L1 miss cezası üzerinden kurulur. Yazmada write-through ana belleği her seferinde günceller, write-back ise dirty biti tutup ana belleği blok değiştirilirken günceller. Gelecek hafta konu virtual memory olacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.